

2

Data Structures in Python

- Lists, Tuples, Sets & Dictionaries
- Built-in methods for organizing and storing data



What are Data Structures?

- A Data Structure is a way of organizing and storing data so it can be accessed and modified efficiently.
- Python provides several built-in data structures that make programming easier.
- The main ones are: List, Tuple, Set, Dictionary and String.

Main Data Structures

Data Structure	Mutable	Ordered	Duplicate Values	Syntax
List	Yes	Yes	Yes	[]
Tuple	No	Yes	Yes	()
Set	Yes	No	No	{ }
Dictionary	Yes	Yes	Keys: No, Values: Yes	{key: value}
String	No	Yes	Yes	" ", ' '

1. Lists — Definition & Features

- A List is an ordered, mutable collection that can store multiple values of different data types.

Features

- Ordered and mutable (can change)
- Allows duplicates
- Supports indexing and slicing
- Can contain different data types

Lists — Syntax & Example

A list is created using square brackets.



main.py

```
1 #list_name = [item1, item2, item3]
2
3 fruits = ["Apple", "Banana", "Mango"]
4 print(fruits)
```

Output / Note: ['Apple', 'Banana', 'Mango']

List Indexing

Access list items using their index number, starting from 0.

main.py

```
1 fruits = ["Apple", "Banana", "Mango"]
2
3 print(fruits[0])
4 print(fruits[1])
5 print(fruits[-1])
```

Output / Note:

- Apple
- Banana
- Mango

List Slicing

Syntax: `list[start:stop:step]`

main.py

```
1 numbers = [10, 20, 30, 40, 50, 60]
2
3 print(numbers[1:4])
4 print(numbers[:3])
5 print(numbers[3:])
6 print(numbers[::-1])
```

Output / Note:

- [20,30,40]
- [10,20,30]
- [40,50,60]
- [60,50,40,30,20,10]

Common List Methods

- `append()` — adds one element at the end
- `extend()` — adds multiple elements
- `insert()` — inserts an element at a position
- `remove()` — removes a specific value
- `pop()` — removes the last (or given) element
- `clear()` — removes all elements
- `sort()` — sorts the list
- `reverse()` — reverses the list order
- `count()` — counts occurrences of a value
- `index()` — returns the index of a value

append() & extend()

append() adds one element; extend() adds multiple elements at once.

main.py

```
1 numbers = [1, 2, 3]
2 numbers.append(4)
3 print(numbers)
4
5 a = [1, 2]
6 a.extend([3, 4])
7 print(a)
```

Output / Note:

- [1,2,3,4]
- [1,2,3,4]

insert()

Inserts an element at a specific position in the list.

main.py

```
1 names = ["Ali", "Ahmed"]  
2  
3 names.insert(1, "Sara")  
4 print(names)
```

Output / Note: ['Ali', 'Sara', 'Ahmed']

remove() & pop()

remove() deletes a value; pop() removes the last element by default.

main.py

```
1 fruits = ["Apple", "Banana", "Mango"]
2 fruits.remove("Banana")
3 print(fruits)
4
5 numbers = [10, 20, 30]
6 numbers.pop()
7 print(numbers)
```

Output / Note:

- ['Apple', 'Mango']
- [10, 20]

clear()

Removes all elements from the list, leaving it empty.

main.py

```
1 numbers = [1, 2, 3]
2
3 numbers.clear()
4 print(numbers)
```

Output / Note: []

sort() & reverse()

sort() arranges elements in order; reverse() flips the current order.

main.py

```
1 numbers = [5, 2, 8, 1]
2 numbers.sort()
3 print(numbers)
4
5 numbers = [1, 2, 3]
6 numbers.reverse()
7 print(numbers)
```

Output / Note:

- [1,2,5,8]
- [3,2,1]

count() & index()

count() counts occurrences; index() finds the position of a value.

main.py

```
1 numbers = [1, 2, 2, 3, 2]
2 print(numbers.count(2))
3
4 colors = ["Red", "Blue", "Green"]
5 print(colors.index("Blue"))
```

Output / Note:

- 3
- 1

Nested Lists

A list can contain other lists as its elements.

main.py

```
1 matrix = [  
2     [1, 2],  
3     [3, 4],  
4     [5, 6]  
5 ]  
6  
7 print(matrix[1][0])
```

Output / Note: 3

2. Tuples — Definition & Features

- A Tuple is an ordered and immutable collection. Once created, its values cannot be changed.

Features

- Ordered
- Immutable
- Allows duplicates
- Faster than lists
- Supports indexing

Tuples — Syntax, Indexing & Slicing

A tuple is created using round brackets.

main.py

```
1 colors = ("Red", "Green", "Blue")
2 print(colors)
3
4 print(colors[0])
5 print(colors[-1])
6 print(colors[1:])
```

Output / Note:

- ('Red','Green','Blue')
- Red
- Blue
- ('Green','Blue')

Tuple Packing & Unpacking

Packing groups values into a tuple; unpacking assigns them back to variables.

main.py

```
1 student = ("Ali", 18, "A")    # packing
2
3 name, age, grade = student    # unpacking
4 print(name)
5 print(age)
6 print(grade)
```

Output / Note:

- Ali
- 18
- A

Tuple Methods: count() & index()

Tuples only support two built-in methods.

main.py

```
1 numbers = (1, 2, 2, 3)
2 print(numbers.count(2))
3
4 numbers = (5, 8, 9)
5 print(numbers.index(8))
```

Output / Note:

- 2
- 1

3. Sets — Definition & Features

- A Set is an unordered collection of unique values.

Features

- Unordered
- Mutable
- No duplicates
- No indexing

Sets — Syntax & Example

Duplicate values are automatically removed when a set is created.

main.py

```
1 numbers = {1, 2, 3, 3, 2}
2
3 print(numbers)
```

Output / Note: {1, 2, 3}

add() & remove()

add() inserts an element; remove() deletes a value (raises an error if missing).

main.py

```
1 numbers = {1, 2, 3}
2 numbers.add(4)
3 print(numbers)
4
5 numbers.remove(2)
6 print(numbers)
```

Output / Note:

- {1,2,3,4}
- {1,3,4}

discard(), pop() & clear()

discard() does not raise an error if the value doesn't exist; pop() removes a random element.

main.py

```
1 numbers = {1, 3, 4}
2 numbers.discard(10)    # no error, 10 not in set
3 print(numbers)
4
5 numbers.pop()          # removes a random element
6 numbers.clear()
7 print(numbers)
```

Output / Note:

- {1,3,4}
- set()

Union & Intersection

| combines both sets; & keeps only the common elements.

main.py

```
1 A = {1, 2, 3}
2 B = {3, 4, 5}
3
4 print(A | B)    # union
5 print(A & B)    # intersection
```

Output / Note:

- {1,2,3,4,5}
- {3}

Difference & Symmetric Difference

- keeps values only in the first set; ^ keeps values not shared by both.

main.py

```
1 A = {1, 2, 3}
2 B = {3, 4, 5}
3
4 print(A - B)      # difference
5 print(A ^ B)      # symmetric difference
```

Output / Note:

- {1,2}
- {1,2,4,5}

4. Dictionaries — Definition & Features

- A Dictionary stores data as Key : Value pairs.

Features

- Mutable
- Ordered
- Keys must be unique
- Values can repeat

Dictionaries — Syntax & Example

A dictionary is created using curly braces with key:value pairs.

main.py

```
1 student = {  
2     "name": "Ali",  
3     "age": 18,  
4     "marks": 90  
5 }  
6 print(student)
```

Accessing Values

Use square brackets or `get()` to access a value by its key.

main.py

```
1 student = {"name": "Ali", "age": 18, "marks": 90}
2
3 print(student["name"])
4 print(student.get("age"))
```

Output / Note:

- Ali
- 18

Add, Update & Delete

Dictionaries can be modified after creation.

main.py

```
1 student = {"name": "Ali", "age": 18}
2
3 student["city"] = "Karachi"      # add
4 student["age"] = 20              # update
5 del student["name"]              # delete
6 print(student)
```

Output / Note: {'age': 20, 'city': 'Karachi'}

keys(), values() & items()

These methods return views of a dictionary's keys, values, or key-value pairs.

main.py

```
1 student = {"name": "Ali", "age": 18, "marks": 90}
2
3 print(student.keys())
4 print(student.values())
5 print(student.items())
```

Output / Note:

- dict_keys(['name', 'age', 'marks'])
- dict_values(['Ali', 18, 90])
- dict_items([('name': 'Ali'), ('age': 18), ('marks': 90)])

pop() & update()

pop() removes a key; update() adds or changes multiple key-value pairs.

main.py

```
1 student = {"name": "Ali", "age": 18, "marks": 90}
2
3 student.pop("age")
4 student.update({"marks": 95})
5 print(student)
```

Output / Note: {'name': 'Ali', 'marks': 95}

Nested Dictionary

A dictionary can contain other dictionaries as its values.

main.py

```
1 students = {  
2     "student1": {  
3         "name": "Ali",  
4         "marks": 90  
5     }  
6 }  
7  
8 print(students["student1"]["name"])
```

Output / Note: Ali

Data Structures (1/3) — Multiple Choice Questions

1. Which data structure is ordered and mutable but does not allow key-based access?

A) List B) Set C) Dictionary D) Tuple

2. Which data structure does NOT allow duplicate values?

A) List B) Tuple C) Set D) Dictionary values

3. Which symbol is used to create a tuple?

A) [] B) {} C) () D) <>

4. Which method adds a single element to the end of a list?

A) extend() B) append() C) insert() D) add()

5. What does numbers[::-1] do to a list?

A) Sorts it B) Reverses it C) Clears it D) Copies first half

Answer Key: 1-A 2-C 3-C 4-B 5-B

Data Structures (2/3) — Multiple Choice Questions

1. Which method removes all elements from a list?

A) remove() B) pop() C) clear() D) delete()

2. Which method inserts an element at a specific index?

A) append() B) insert() C) extend() D) add()

3. Tuples are best described as:

A) Mutable and ordered B) Immutable and ordered C) Mutable and unordered D) Immutable and unordered

4. Which set method does NOT raise an error if the value is missing?

A) remove() B) discard() C) pop() D) clear()

5. Which operator returns the union of two sets?

A) & B) - C) | D) ^

Answer Key: 1-C 2-B 3-B 4-B 5-C

Data Structures (3/3) — Multiple Choice Questions

1. A dictionary stores data as:

- A) Ordered values only B) Key-value pairs C) Unique numbers
D) Nested lists

2. Which method safely retrieves a value from a dictionary?

- A) get() B) fetch() C) find() D) access()

3. Which method removes a specific key from a dictionary?

- A) remove() B) delete() C) pop() D) clear()

4. Which method returns all key-value pairs of a dictionary?

- A) keys() B) values() C) items() D) pairs()

5. In a dictionary, which of these must be unique?

- A) Values B) Keys C) Both keys and values D) Neither

Answer Key: 1-B 2-A 3-C 4-C 5-B

3

Python Control Flow

- if, elif, else & nested if
- match-case statement

What is Control Flow?

- Control Flow is the order in which Python executes the statements in a program.
- It decides which statement runs first and which runs afterward.
- By default, every Python program executes from top to bottom, one statement at a time.

Control Flow — Example

Without any conditions, statements simply run in the order they are written.

main.py

```
1 print("Start")
2 print("Learning Python")
3 print("End")
```

Output / Note:

- Start
- Learning Python
- End

Why is Control Flow Important?

- In real life, almost every task depends on a condition — and Python programs work the same way.

Real-Life Examples

- ATM → if the password is correct, withdraw money; otherwise, show an error.
- School → if marks ≥ 40 , the student passes; otherwise, they fail.
- Website Login → check the username, then the password, then allow login.

Decision Making

- Decision making means the program checks a condition and takes action based on the result.
- Example: `age = 20` → `is age >= 18?` → Yes: can vote, No: cannot vote.

Python uses these tools for decision making:

- `if`
- `elif`
- `else`
- `match-case`

The if Statement

Definition

- The if statement checks a condition.
- If the condition is True, its block of code executes.
- If the condition is False, the block is skipped.

Syntax

- if condition:
- statement

Important

- A colon (:) is always required after the condition.
- The block must be indented (4 spaces).

if — Example: Voting Age

The block only runs when the condition evaluates to True.

main.py

```
1 age = 20
2
3 if age >= 18:
4     print("You can vote.")
```

Output / Note: You can vote.

if — More Examples

if statements can check numeric conditions like marks or temperature.

main.py

```
1 marks = 85
2 if marks >= 40:
3     print("Pass")
4
5 temperature = 45
6 if temperature > 40:
7     print("It is very hot today.")
```

Output / Note:

- Pass
- It is very hot today.

Multiple Statements Inside if

An if block can contain more than one statement — all run together when True.

main.py

```
1 salary = 50000
2
3 if salary >= 40000:
4     print("Eligible")
5     print("Apply Now")
6     print("Good Salary")
```

Output / Note:

- Eligible
- Apply Now
- Good Salary

if — Using User Input

Combine `input()` with `if` to make decisions based on what the user types.

main.py

```
1 age = int(input("Enter your age: "))
2
3 if age >= 18:
4     print("Eligible for CNIC")
```

Output / Note:

- Enter your age: 18
- Eligible for CNIC

What is else?

Definition

- The else statement executes a block of code when the if condition is False.
- If the if condition fails, the else block runs instead.

Syntax

- if condition:
 - # code if condition is True
- else:
 - # code if condition is False

else — Pass or Fail

else provides a fallback path when the condition is not met.

main.py

```
1 marks = 35
2
3 if marks >= 40:
4     print("Pass")
5 else:
6     print("Fail")
```

Output / Note: Fail

else — Voting Eligibility

The else block runs whenever the if condition evaluates to False.



main.py

```
1 age = 15
2
3 if age >= 18:
4     print("Eligible to Vote")
5 else:
6     print("Not Eligible")
```

Output / Note: Not Eligible

else — Even or Odd

The modulus operator (%) is commonly used with if-else to check even/odd.

main.py

```
1 number = 9
2
3 if number % 2 == 0:
4     print("Even")
5 else:
6     print("Odd")
```

Output / Note: Odd

else — ATM Withdrawal

if-else is ideal for simple approve/reject decisions like withdrawals.

main.py

```
1 balance = 5000
2 withdraw = 7000
3
4 if withdraw <= balance:
5     print("Transaction Successful")
6 else:
7     print("Insufficient Balance")
```

Output / Note: Insufficient Balance

What is elif?

Definition

- elif means "else if" and is used when there are multiple conditions to check.
- If the first condition is False, Python checks the next elif condition, and so on.

Syntax

- if condition1:
 - statement
- elif condition2:
 - statement
- else:
 - statement

elif — Student Grade

Python checks each condition in order and stops at the first True one.

```
main.py
1 marks = 82
2
3 if marks >= 90:
4     print("Grade A+")
5 elif marks >= 80:
6     print("Grade A")
7 elif marks >= 70:
8     print("Grade B")
9 else:
10    print("Fail")
```

Output / Note: Grade A

elif — Traffic Light

elif works well for checking a variable against several exact values.



main.py

```
1 signal = "Yellow"
2
3 if signal == "Red":
4     print("Stop")
5 elif signal == "Yellow":
6     print("Ready")
7 elif signal == "Green":
8     print("Go")
9 else:
10    print("Invalid Signal")
```

Output / Note: Ready

elif — Temperature

elif chains can handle several numeric ranges cleanly.

main.py

```
1 temp = 42
2
3 if temp >= 45:
4     print("Extreme Heat")
5 elif temp >= 35:
6     print("Hot")
7 else:
8     print("Normal / Cold")
```

Output / Note: Hot

elif — Online Shopping Discount

elif can map numeric ranges to different outcomes, like discount tiers.



main.py

```
1 amount = 6000
2
3 if amount >= 10000:
4     print("30% Discount")
5 elif amount >= 5000:
6     print("20% Discount")
7 else:
8     print("10% or No Discount")
```

Output / Note: 20% Discount

Difference: if, elif, else

Statement	Purpose
if	Checks the first condition.
elif	Checks another condition if the previous one is False.
else	Executes when all previous conditions are False.



Rules of if-elif-else

- if always comes first.
- elif can appear zero or multiple times.
- else can only appear once.
- else always comes at the end.

What is Nested if?

Definition

- A Nested if means an if statement written inside another if statement.

Why Do We Use Nested if?

- Nested if is used when a second condition should only be checked after the first condition is True.

Real-Life Example: ATM Machine

- First check: Is the card inserted?
- Then check: Is the PIN correct?
- Only then: Withdraw money.

Nested if — Syntax & Voting Example

The inner if only runs when the outer if condition is True.

main.py

```
1 age = 20
2 citizen = True
3
4 if age >= 18:
5     if citizen:
6         print("Eligible to Vote")
```

Output / Note: Eligible to Vote

Nested if — ATM & Scholarship

Both conditions must be True for the inner block to run.

main.py

```
1 pin, balance = 1234, 10000
2 if pin == 1234:
3     if balance >= 5000:
4         print("Withdrawal Allowed")
5
6 marks, attendance = 85, 90
7 if marks >= 80:
8     if attendance >= 75:
9         print("Scholarship Granted")
```

Output / Note:

- Withdrawal Allowed
- Scholarship Granted

Nested if — Login System

A common real-world use: verifying a username, then a password.

main.py

```
1 username = "admin"
2 password = "1234"
3
4 if username == "admin":
5     if password == "1234":
6         print("Login Successful")
```

Output / Note: Login Successful

Nested if-else

- A nested if can also contain its own else, giving more precise decision paths.

Syntax

- if condition1:
 - if condition2:
 - statement
 - else:
 - statement
- else:
 - statement

Nested if-else — Bank Loan

The else can attach to either the outer or the inner if.

main.py

```
1 salary = 70000
2 experience = 3
3
4 if salary >= 50000:
5     if experience >= 2:
6         print("Loan Approved")
7     else:
8         print("Need More Experience")
9 else:
10    print("Low Salary")
```

Output / Note: Loan Approved

Nested if-else — Admission & Promotion

Nested if-else lets each condition have its own pass/fail outcome.



main.py

```
1 percentage, entry_test = 78, 65
2 if percentage >= 70:
3     if entry_test >= 60:
4         print("Admission Confirmed")
5     else:
6         print("Entry Test Failed")
7 else:
8     print("Low Percentage")
```

Output / Note: Admission Confirmed

Nested if — Advantages & Disadvantages

Advantages

- Easy for dependent conditions.
- Enables better decision making.
- Suitable for security systems, banking software, and login systems.

Disadvantages

- Too many nested levels make code difficult to read.
- Large nested code is difficult to maintain.

What is match-case?

Definition

- match-case is Python's version of a switch statement.
- It compares one value against multiple possible cases.
- Available from Python 3.10 onward.

Why Use match-case?

- Instead of writing many elif statements for the same variable, match-case offers a cleaner alternative for comparing one value against several fixed options.

match-case — Syntax

- match variable:
 - case value1:
 - statement
 - case value2:
 - statement
 - case _:
 - statement
-
- The underscore (_) means the default case, similar to else.

match-case — Day Name

match-case compares a single value against several fixed cases.



main.py

```
1 day = 3
2 match day:
3     case 1:
4         print("Monday")
5     case 2:
6         print("Tuesday")
7     case 3:
8         print("Wednesday")
9     case _:
10        print("Invalid Day")
```

Output / Note: Wednesday

match-case — Simple Calculator

Each case handles one exact operator value.



main.py

```
1 operator = "+"
2
3 match operator:
4     case "+":
5         print(10 + 5)
6     case "-":
7         print(10 - 5)
8     case _:
9         print("Invalid Operator")
```

Output / Note: 15

match-case — Menu Selection

match-case is especially clean for menu-driven programs.

```
main.py

1 choice = 2
2 match choice:
3     case 1:
4         print("Burger")
5     case 2:
6         print("Pizza")
7     case 3:
8         print("Biryani")
9     case _:
10        print("Invalid Choice")
```

Output / Note: Pizza

match-case — Grade

case values can also be strings, not just numbers.

main.py

```
1 grade = "A"
2
3 match grade:
4     case "A":
5         print("Excellent")
6     case "B":
7         print("Very Good")
8     case _:
9         print("Needs Improvement")
```

Output / Note: Excellent

Difference: if-elif vs match-case

if-elif	match-case
Can compare different conditions	Compares one value
Supports logical operators	Mainly exact value matching
More flexible	Cleaner for menu-based programs
Available in all Python versions	Python 3.10+ only

When to Use Which?

Use if-elif when:

- Comparing ranges (marks $\geq$ 80)
- Using logical operators (and, or)
- Comparing multiple variables

Use match-case when:

- Creating menus
- Handling user choices
- Mapping fixed values (days, months, grades)



Control Flow — Best Practices

- Use nested if only when conditions depend on each other.
- Avoid deep nesting (more than 2-3 levels).
- Use match-case for menu-driven programs.
- Use meaningful variable names.

Control Flow (1/3) — Multiple Choice Questions

1. By default, Python executes statements:

- A) Bottom to top B) Randomly C) Top to bottom D) All at once

2. What is required after an if condition?

- A) A semicolon B) A colon C) Curly braces D) Nothing

3. Which block runs when an if condition is False and no elif/else exists?

- A) The if block still runs B) Nothing runs C) Python raises an error
D) The program restarts

4. Which statement provides a fallback when the if condition is False?

- A) elif B) else C) case D) pass

5. How many else blocks can a single if-elif-else chain have?

- A) 0 B) 1 C) 2 D) Unlimited

Answer Key: 1-C 2-B 3-B 4-B 5-B

Control Flow (2/3) — Multiple Choice Questions

1. elif stands for:

A) Else Loop B) Else If C) End If D) Extra If

2. How many elif blocks can one if statement have?

A) Exactly 1 B) Exactly 2 C) Zero or more D) None allowed

3. A Nested if is:

A) An if inside a loop B) An if inside another if C) Two separate if statements D) An if with no condition

4. Nested if is most suitable for:

A) Single simple conditions B) Dependent conditions C) Loops only D) String formatting

5. A key disadvantage of deep nesting is:

A) Faster execution B) Reduced readability C) No conditions allowed D) Cannot use else

Answer Key: 1-B 2-C 3-B 4-B 5-B

Control Flow (3/3) — Multiple Choice Questions

1. match-case is available from which Python version?

A) 3.6+ B) 3.8+ C) 3.10+ D) 2.7+

2. In match-case, which symbol represents the default case?

A) * B) _ C) ? D) default

3. match-case is best suited for:

A) Range comparisons B) Logical operator conditions C) Comparing one value against fixed cases D) Loops

4. Which is available in all Python versions?

A) match-case B) if-elif C) Both equally D) Neither

5. Which should be avoided for readability?

A) Meaningful variable names B) Deep nested if levels C) Using else D) Using elif

Answer Key: 1-C 2-B 3-C 4-B 5-B



Thank You!

Keep practicing — write code every day to master Python.